

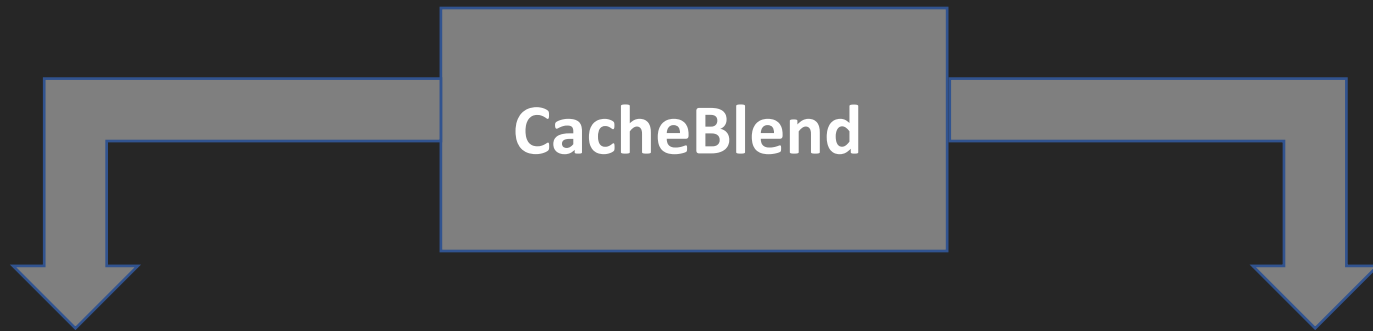


Artificial Intelligence 101

CacheBlend: Bridging Speed & Efficiency In Data Management

Ctrl C + Ctrl V Crew

CacheBlend Vs Existing methods



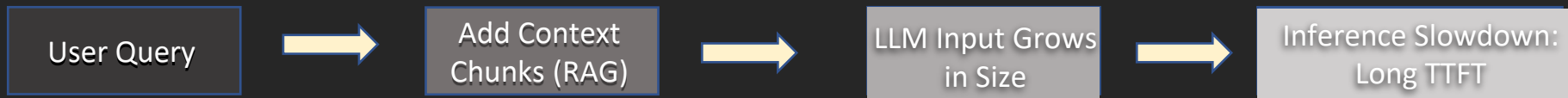
What is the problem?

- 1) LLM inference slows down significantly when multiple context chunks are added.
- 2) Existing methods to reuse precomputed KV caches are inefficient:
 - **Prefix caching** only works for single-chunk prefixes.
 - **Full KV reuse** ignores critical cross-attention between chunks, leading to poor generation quality.

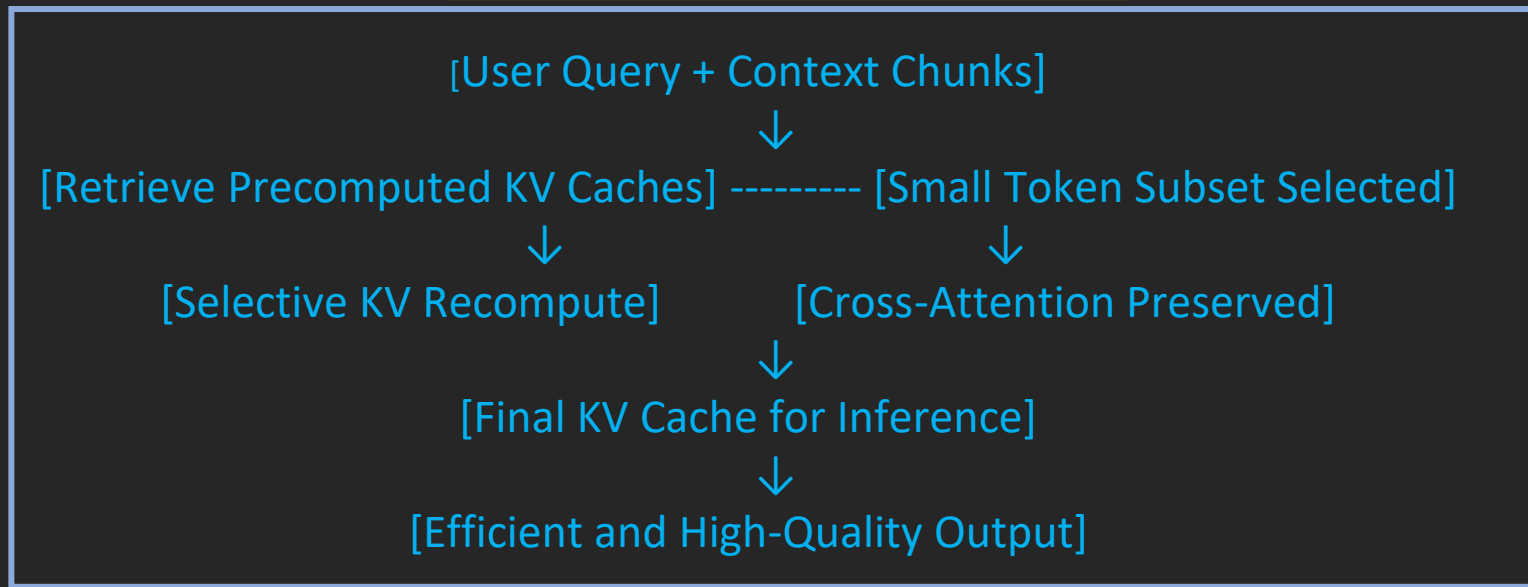
What is CacheBlend?

- 1) A novel system that combines precomputed KV caches for all text chunks, not just prefixes.
- 2) Selectively **recomputes KV values for a small subset of tokens**.
- 3) Pipelining reduces delays even when storage is on slower devices.

Current Challenges in LLM Inference



How CacheBlend Works?



CacheBlend Vs Existing Methods

Feature	Full KV Recompute	Prefix Caching	Full KV Reuse	CacheBlend (Proposed)
Recomputes All Tokens?	Yes	Partial	No	No(Selective Recompute)
Cross-Attention Preserved?	Yes	Yes	No	Yes
TTFT	High	Moderate	Low	Low
Quality	High	High	Low	High
Storage Cost	High	Moderate	Low	Low

BACKGROUND

An outline of the existing LLM Services &
their limitations

PREFILL PHASE

Token
Embeddings

Q, K, V vectors

Attention
Matrix(Q.K)

Normalization
Masking

(Attention Matrix).
V

KV Cache

- K1, V1
- K2, V2
- ..
- Kn, Vn

KV Cache

- K1, V1
- K2, V2
- ..
- Kn, Vn
- Knew, Vnew

DRAWBACKS

```
graph TD; A[DRAWBACKS] --> B[Slow Response Time]; A --> C[High Computational Cost];
```

Slow Response Time

High Computational Cost

Motivation

(How cache can be used?)

CHUNK REUSING

KV pairs for
each token



Prefill
Overhead

*Who in the IT department proposed
using RAG to enhance customer
service during the last all-hands
meeting?"*

*Who from the IT department were
graduates from college Y?"*

KV Pairs of IT
department

*Instead of recomputing KV
pairs again, we can REUSE IT*



PREFIX CACHING & WHY IT SUCKS!

Text chunks at the **START** that are likely to be reused are cached.



When required to be used again, we use the cache again.



Doraemon is my 2nd favorite cartoon show. He is a cat robot who has superpowers. He lives at nobita's house. Nobita is a very lazy character but innocent. He always gets himself in trouble, and whenever he is in trouble, Doraemon will be there to help him. Nobita has a female friend named Shizuka. Nobita has several enemies, too, Sumio and Jian. They are best friends but still bully nobita. They always put nobita in



Any important text chunk later is not cached.

Disadvantages

- **Only the first chunk** (the prefix) can have its cache reused.
- Other chunks that come after it do not benefit from prefix caching, even if they are repeated elsewhere in the input.
- This is a limitation in scenarios where multiple context chunks are needed to answer a query.

Advantages

- ✓ Unaffected by the succeeding text in the input, meaning the result generated by the model (after applying the prefix cache)
- ✓ It will be identical to the result you'd get if you recomputed the KV cache for that prefix from scratch.
- ✓ Thus, it **maintains the ACCURACY**.

FULL KV REUSE

It precomputes KV pairs of all multiple text chunks

C1: The quick brown

+ BUFFERS

C2: jumped over the lazy dog

CONCATENATION

Chunk 1

"Lionel Messi scored 13 goals at FIFA World Cups.\n"

Chunk 2

"Cristiano scored 8 goals at FIFA World Cups.\n"

Query

"Who scored more goals at FIFA World Cups, Messi or Ronaldo?\n"

KV cache
Chunk 1

KV cache
Chunk 2

+ Query

LLM

"The question is asking for information about FIFA World Cups. The names of Messi and Ronaldo are well-known ..."

(c) Full KV reuse gives wrong answer.

✓ But this **ignores the CROSS ATTENTION** between C1 & C2.

✓ Since full KV reuse precomputes each chunk separately, the cross attention between two chunks is completely missed (never computed) when the KV caches are precomputed

FULL KV RECOMPUTE

No need of cache.

It recalculates the KV pairs for the required tokens.

Since all KVs are recomputed from SCRATCH, it is slower

But turns out to be more **ACCURATE!**

Chunk 1

"Lionel Messi scored 13 goals at FIFA World Cups.\n"

Chunk 2

"Cristiano scored 8 goals at FIFA World Cups.\n"

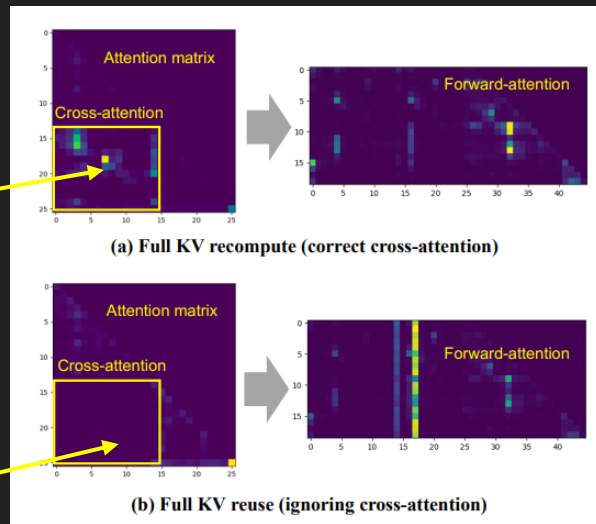
Query

"Who scored more goals at FIFA World Cups, Messi or Ronaldo?\n"



(b) Full KV recompute gives correct answer.

Proper attention taken care of...



Nothing!

Advancing CacheBlend Optimization

Methodology and Evaluation

Models

Evaluated on various models including LLaMA, Llama-2, Llama-3, and Mistral.

Metrics

Measured perplexity on Wikitext-2 and C4 datasets using teacher forcing.

Comparisons

Compared against uniform quantization, nonuniform quantization, Atom, and FlexGen.

Performance on Wikitext-2

Consistent Outperformance

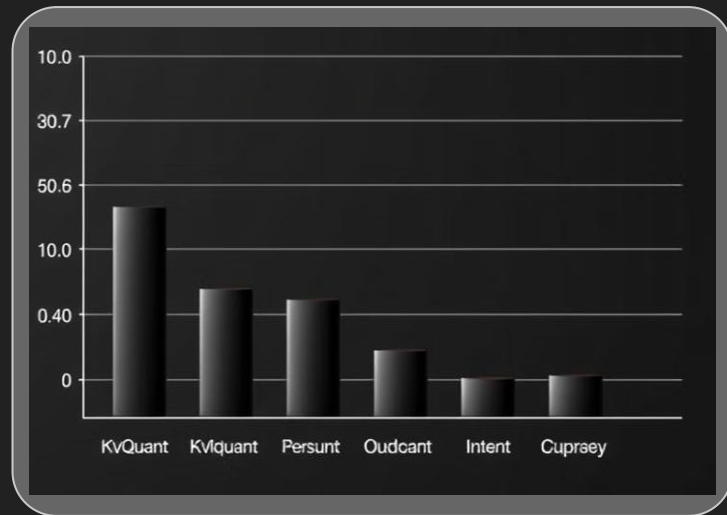
KVQuant consistently outperforms baseline approaches, especially with 3-bit and 2-bit quantization.

Low-Precision Gains

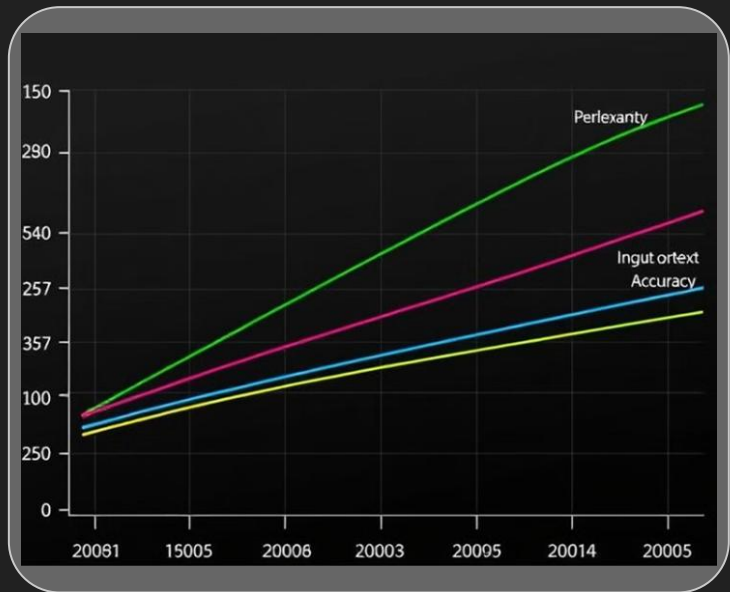
Achieves 4-bit quantization with minimal degradation, 3-bit with under 0.1 degradation, and 2-bit with under 0.5 degradation.

Memory Savings

Attains 3.7×, 4.8×, and 6.9× memory savings for 4-bit, 3-bit, and 2-bit quantization, respectively.



Long Context Length Evaluation



1

Perplexity Evaluation

Evaluated on LLaMA-2-7B32K and Llama-2-70B-32K LongLoRA models.

2

Passkey Retrieval

Evaluated on passkey retrieval tasks to assess the model's ability to locate specific information in long texts.

3

LongBench Evaluation

Evaluated on LongBench, a suite of long-context length evaluation benchmarks.

4

RULER Evaluation

Evaluated on the RULER benchmark suite, demonstrating KVQuant's effectiveness for long-context tasks.

Tested KV cache quantization combined with weight quantization from SqueezeLLM

Joint Weight and KV Cache Quantization



Minimal Degradation

Minimal perplexity degradation observed when quantizing both weights and KV cache.



Compatibility

KVQuant is compatible with existing weight-only quantization methods like SqueezeLLM.

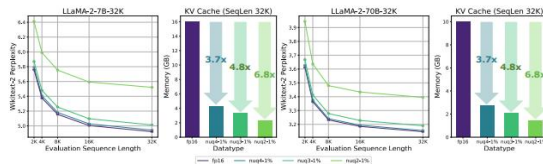


Figure 3: Perplexity results for the LLaMA-2-7B-32K model [5] as well as the Llama-2-70B-32K LongLoRA model [6] on the Wikitext-2 dataset, evaluated using different sequence lengths.

Table 2: Passkey retrieval results across different context lengths for the LLaMA-2-7B-32K model (uptrained for long sequence lengths using positional interpolation [5]) as well as the Llama-2-70B-32K LongLoRA model [6]. The values reported are the success rate for retrieving the passkey, computed over 50 samples. We also include comparisons with KIVI for reference, using the 2-bit configuration with group size of 32 and 128-element fp16 residual [26]. Average bit widths are estimated for each approach assuming 32K context length. Note that the open-source code for running KIVI with LLaMA does not support grouped-query attention, so we did not include comparisons with KIVI for Llama-2-70B-32K.

Model	Method	2K	4K	8K	16K	32K	Avg. Bit Width
LLaMA-2-7B-32K	fp16	1	1	1	1	1	16
	KIVI-2-gs32-r128	0.76	0.72	0.72	0.68	0.7	3.05
	m4-1%	1	1	1	1	1	4.33
	m4-3-1%	0.98	1	1	1	1	3.33
	m4-2-1%	1	1	0.98	1	1	2.33
Llama-2-70B-32K	fp16	1	1	1	1	1	16
	m4-1%	1	1	1	1	1	4.35
	m4-3-1%	1	1	1	1	1	3.35
	m4-2-1%	0.98	0.98	0.96	1	0.74	2.35

Performance Analysis & Memory Savings

1.2–1.6×

Key Multiplication

Latency savings achieved for
key multiplications.

1.3–1.7×

Value Multiplication

Latency savings achieved for
value multiplications.

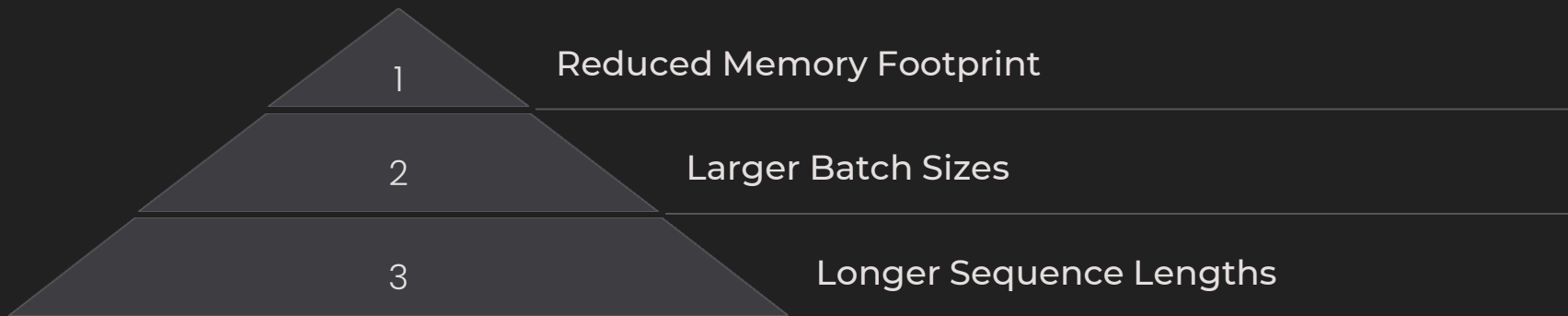
Table 3: LongBench evaluation for the LLaMA-2-7B-32K model using KVQuant-3bit-1%. Comparisons with KIVI are included for reference, using the configuration with group size of 32 and 128-element fp16 residual [26]. Average bit widths are estimated for each approach assuming 12.2K context length, which was the average number of tokens across all tasks.

Config	Avg. bit	NumQA	Qwen	Mt-on	Hotpot	2Wiki	Musique	GovRep	QNLI	Mews	TREC	TriviaQA	SQuAD	RTE	LCC	PassRet	PassCat	Avg.
fp16 Baseline	16	17.96	10.51	33.43	12.55	12.53	6.19	29.65	16.99	22.15	71	87.79	43.97	59.99	62.14	23	1.50	31.96
KIVI-2-gs32-r128	3.17	19.25	10.66	24.78	12.48	11.19	6.38	27.05	16.36	23.37	71	80.80	43.93	57.74	60.61	13.58	1.50	30.04
KVQuant-3bit-1%	3.33	18.87	13.67	30.93	12.07	12.55	6.25	27.10	16.53	16.54	71	87.55	43.95	59.50	61.52	19.5	1.75	31.21

Table 4: RULER evaluation results for the LLaMA-2-7B-32K model with KVQuant quantization methods. We report accuracy across RULER tasks, comparing our KVQuant configurations to baseline and KIVI approaches. A maximum context length of 32K is used for evaluation. Our results show that our method retains baseline accuracy even with aggressive quantization and pruning.

Config	Avg. bit	Ntst1	Ntst2	Ntst3	MKey1	MKey2	MKey3	NValue	MQuery	YT	CWE	FWE	QA1	QA2	Avg.
fp16 Baseline	16	100	99.8	98.6	94	68.2	11	55.95	64.5	37.88	9.64	30.4	31.6	31.6	56.40
KIVI-2-gs32-r128	3.05	76	85.6	59.6	72.6	11.4	0	34.7	46.45	39.6	8.26	30.53	24.8	27.6	39.78
KVQuant-3bit-1%	3.33	99.8	98.8	95.2	92.8	61.6	6.4	47.5	54.45	41.04	8.52	29.33	31.0	31.0	53.65
KVQuant-2bit-1%	2.33	95.4	86.8	49.8	73.6	23.4	0	16.65	22.95	22.52	5.14	24.0	26.4	28.4	36.54

Benefits for Long Context Lengths



Future Directions

1 Significant Compression

KVQuant achieves significant compression while maintaining accuracy, enabling efficient long-context inference.

2 Future Research

Further research will explore extending KVQuant to other model architectures and tasks.



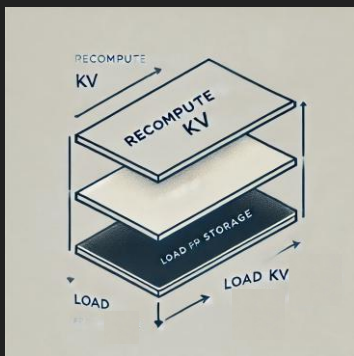
Key Insights and Workflow of CacheBlend

Key Insights behind CacheBlend

CacheBlend reduces delays by

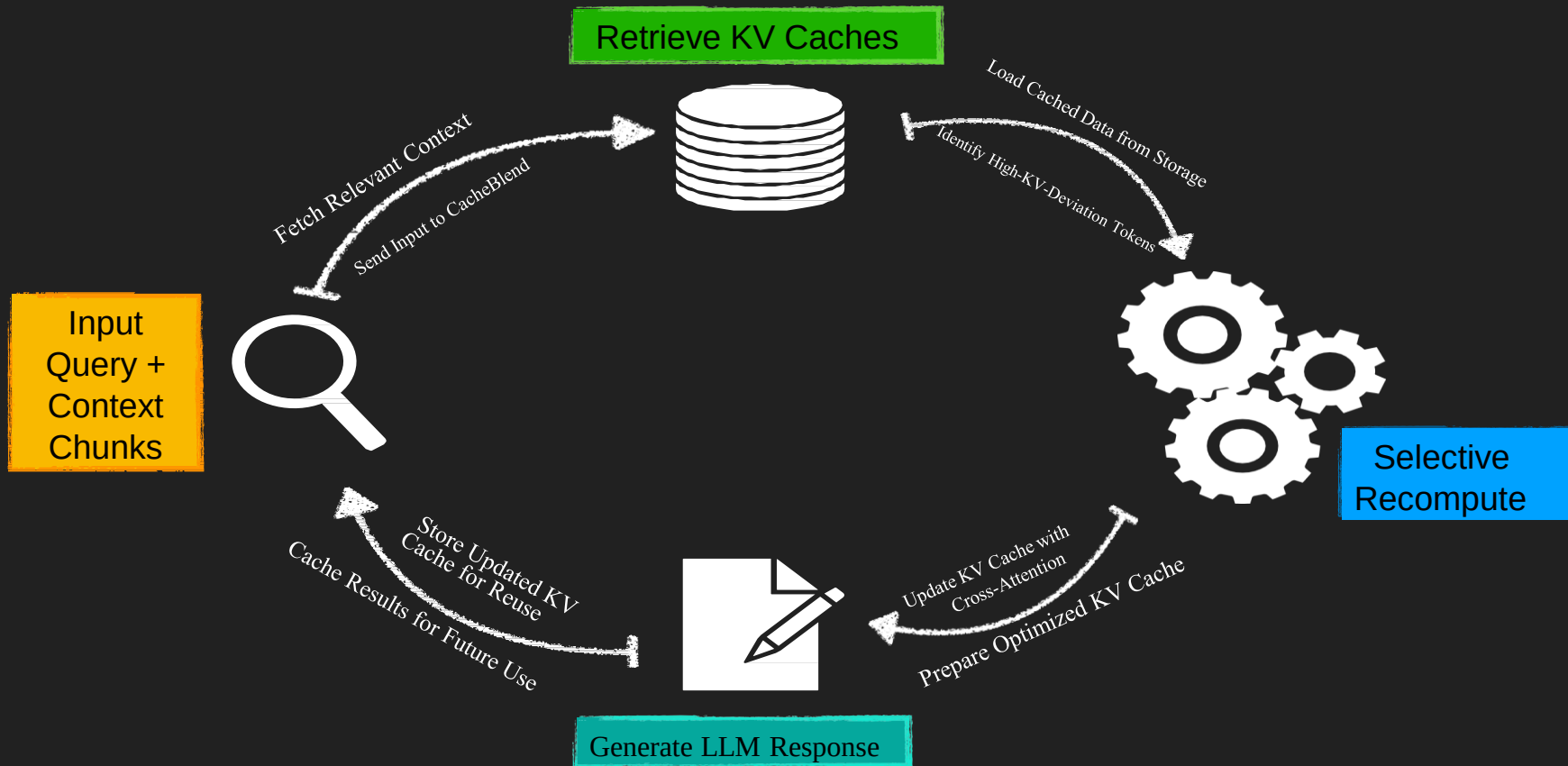
Recomputing selective KV tokens

Loading KV caches into GPU memory



Results
Hides recomputation delays,
improving speed without quality
loss

CacheBlend System Workflow



Components of CacheBlend

Loading Controller

KV Cache Store

Fusor

Loading Controller

- How many tokens to recompute. Where to store/retrieve KV caches for optimal speed and cost.
- Uses delay estimators to balance loading and recompute time.

KV Cache Store

- Manages storage and retrieval of precomputed KV caches.
- Stores caches on RAM, SSD, or slower devices, based on availability.

Fusor

- Combines precomputed KV caches with selective recomputation.
- Ensures cross-attention is updated for high-quality results.

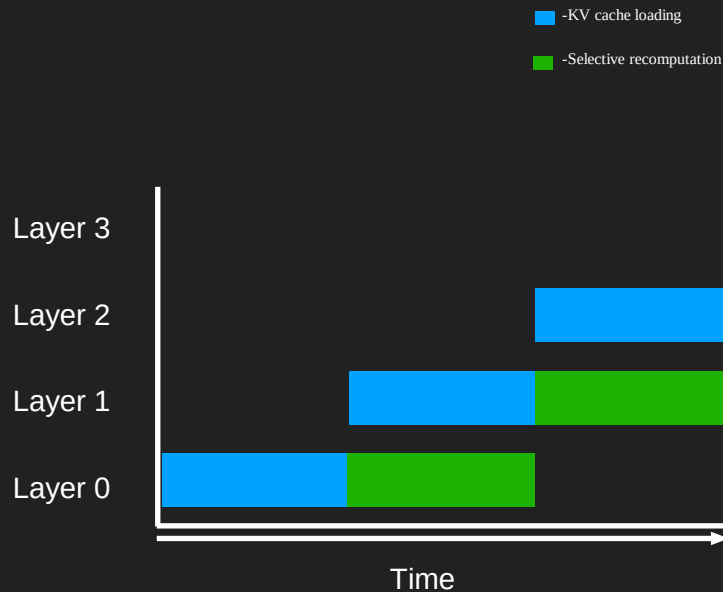
Visual: Block diagram showing interaction between these components.

Pipelining for Efficiency

Three quick steps on how the process works:

1. Fetch KV caches for **Layer 1**.
2. Recompute selective tokens for **Layer 0** simultaneously.
3. Repeat for all layers to overlap the process.

Result: Overlaps KV loading with recomputation, hiding delays.



Benefits of the System

- 1. Fast Inference:** Reduces Time-to-First-Token (TTFT) by 2.2–3.3×
- 2. High Throughput:** Increases throughput by 2.8–5×
- 3. Quality Assurance:** Maintains generation quality with minimal KV updates
- 4. Cost-Efficiency:** Supports storing KV caches on slower, cheaper devices

Implementation

(The Logic)

fetch_kv(text, layer_id)

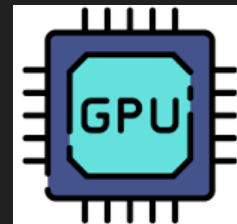
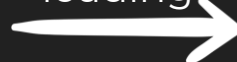
calculate hash
of the text



synchronize()---->(Checkpoint)

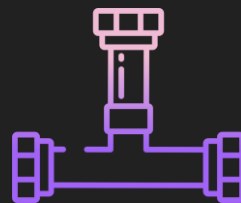
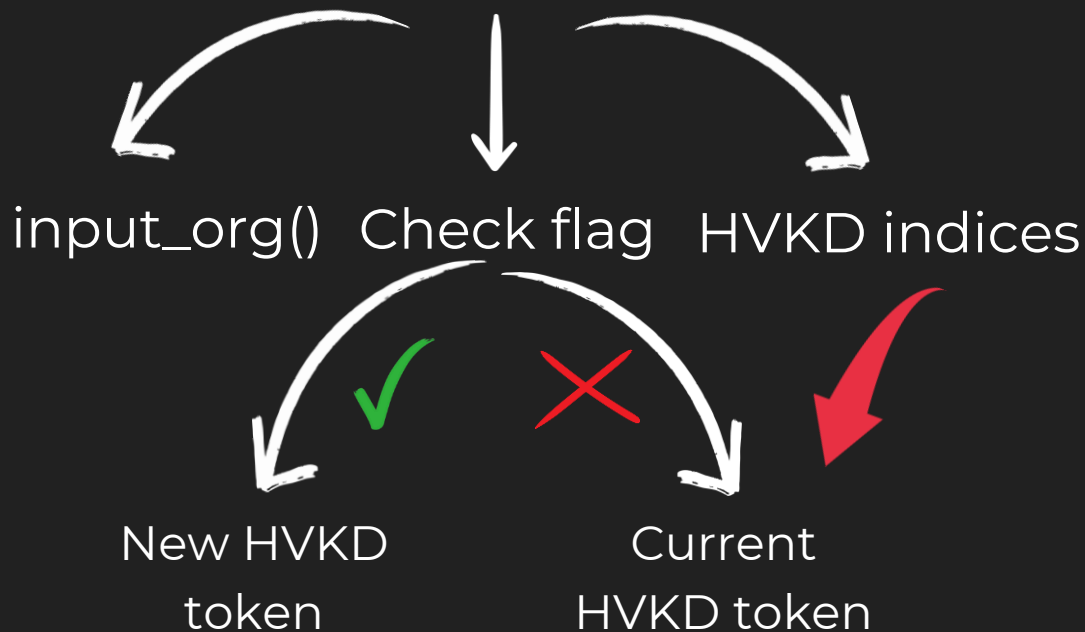


KVCache
loading



Layer i

prefill_layer
(input_dict, KVCache)

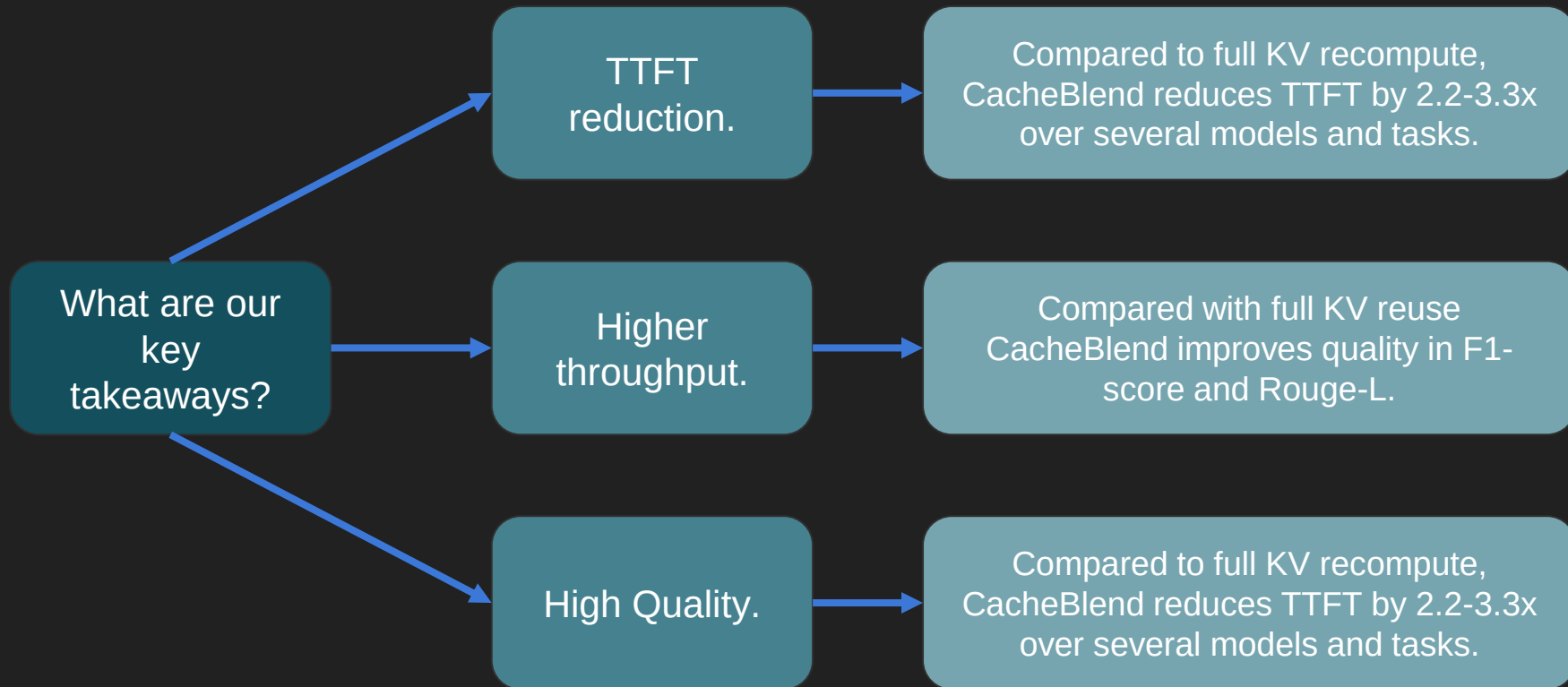


Layer i+1
fetch_kv

If KV cache not found in system, it is recomputed during run time. Moved to CPU using `torch.cpu()` and written to disk using `torch.save()`

Evaluation Framework and Metrics

EVALUATION



SETUP

- Cacheblend was evaluated on 3 popular open-source LLMs: Mistral-7B, Yi-34B, and Llama-70B.
- Used several datasets like 2WikiMQA, Musique, SAMSum, and MultiNews to test model's performance on different tasks
- The experiments were conducted on powerful hardware with various GPUs and 128 GB RAM.

DATASETS:

2WikiMQA: Aims to test LLM's reasoning skills by requiring the model to read multiple paragraphs to answer a given question.

Musique: A multi-document question-answering dataset. Designated to test LLM's multi-hop reasoning ability where one reasoning step critically relies on information from another

SAMSum: comprises multiple pairs of dialogues and summaries, and requires the LLM to output a summary to a new dialogue. It is intended to test the few-shot learning ability of language models

MultiNews: This dataset consists of news articles and human-written summaries of these articles from the site www.newser.com.

QUALITY METRICS

F1-Score is used to evaluate 2WikiMQA and Musique datasets. It measures the similarity between the model's output and the ground-truth answer of the question based on the number of overlapping words.

Rouge-L score is used to evaluate MultiNews and SAMSum datasets. It measures the similarity between the model's output and the ground-truth summaries based on the longest common sequence.

BASELINES

Full KV Recompute

Prefix caching

Full KV reuse

MapReduce

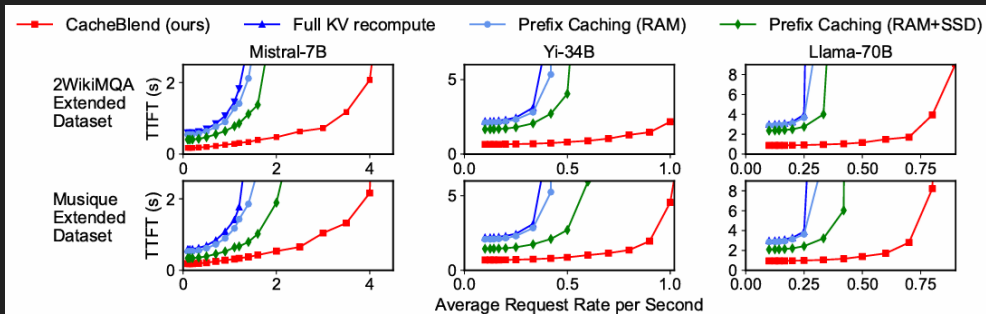
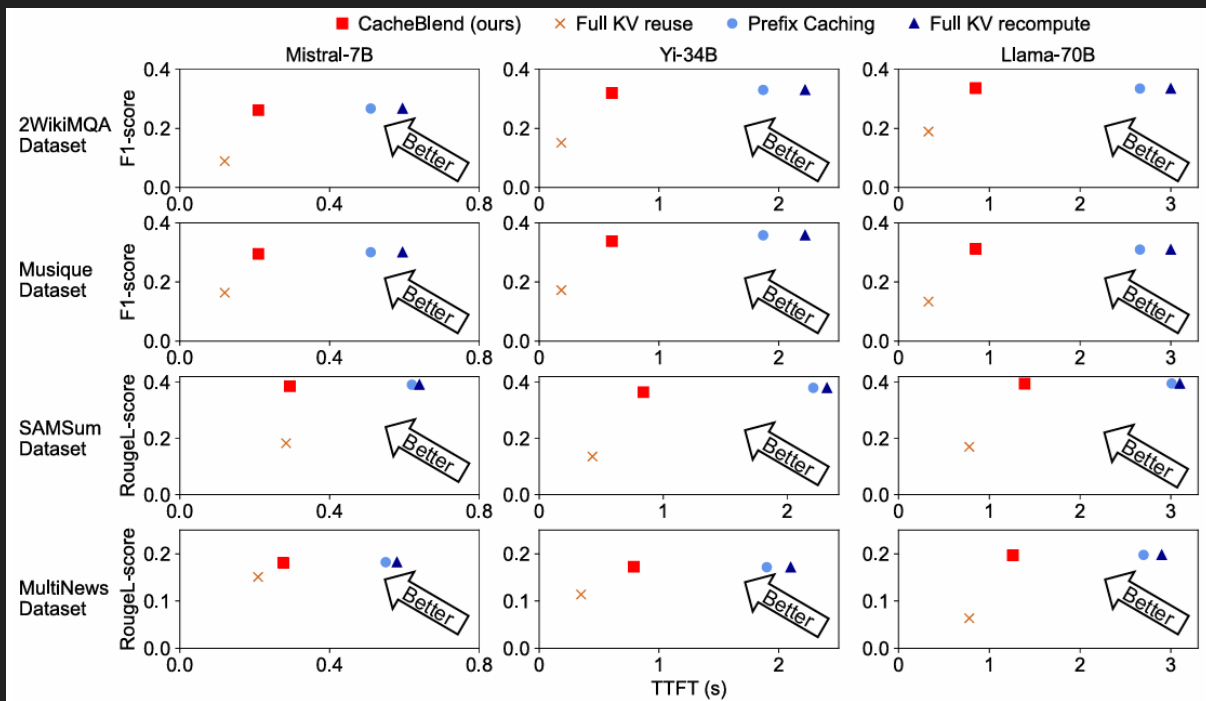
MapRerank

Improvements & Graphs

Overall Improvements

Compared to :	CACHEBLEND
Full KV Recompute	· Lower Delay · Higher throughput
Full KV Reuse	· Better quality due to cross attention
Prefix Caching	· Doesn't need to store multiple versions of KV caches for the same chunk if they have different prefixes
RAG Methods	· Lower Delay · Better Quality

GRAPHS



Optmizing LLM Performance with CacheBlend

(RAG, KV, Cache Reuse & Beyond)

1. Retrieval-Augmented Generation (RAG)

Purpose of RAG

Enhances LLM accuracy and reliability by incorporating external text chunks.

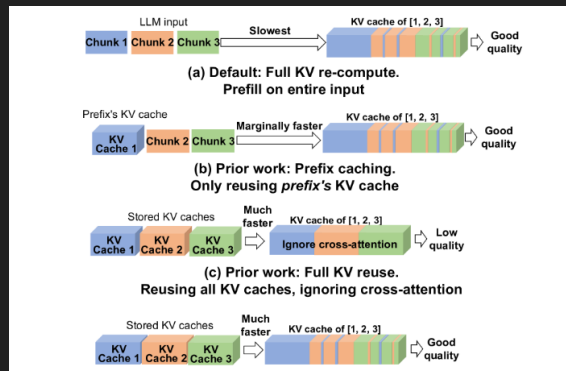
Challenge

Processing these text chunks in LLMs is time-consuming.

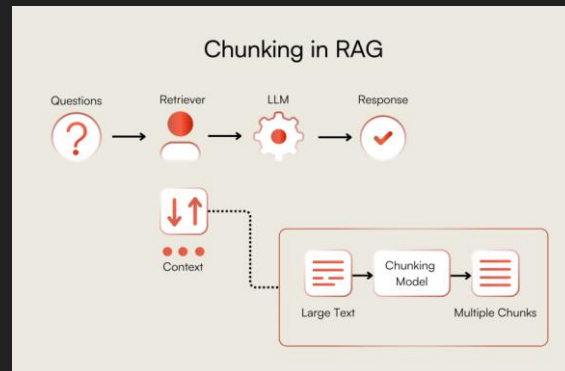
CacheBlend's Solution

Stores and reuses KV caches of text chunk. Significantly reduces overhead.

With Cacheblend

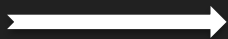


Without Cacheblend



2. KV Cache Reuse :

Existing Approach



Promptcache

Challenges



- > Inaccurate positional encoding.
- > Ignores cross-attention, affecting generation quality.

Cacheblend's Advantage



Positional accuracy and Cross-attention capabilities.

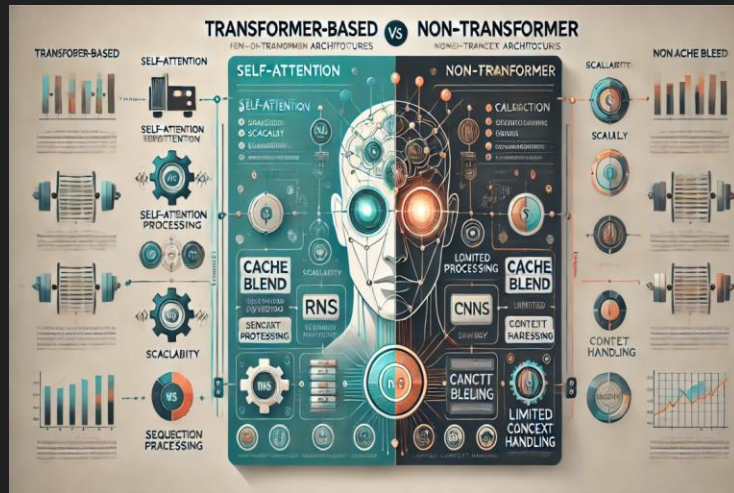
3. Innovations in KV Cache Storage:

4. General-Purpose LLM Serving Systems:

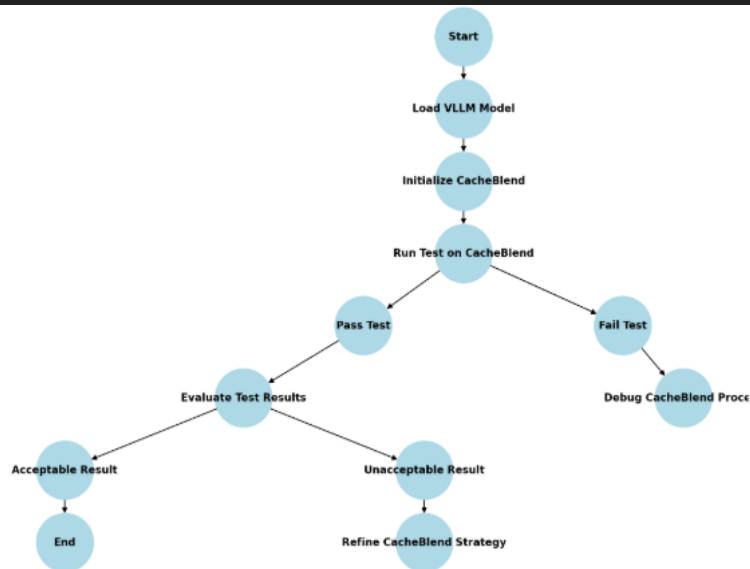
- > Cacheblend empowers the system by context reuse capabilities

Limitations of CacheBlend

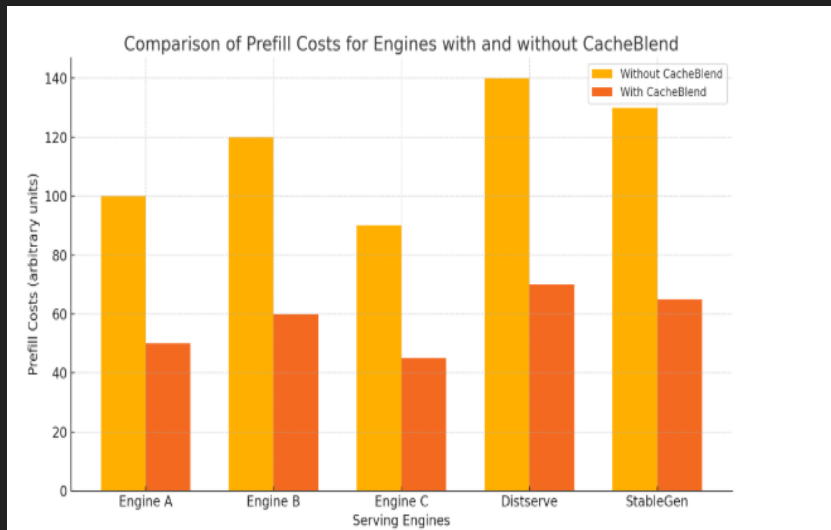
1. Limited applicability to particular transformers only



2. Evaluation restricted to vLLMs



3. Prefill phase optimization is untapped for advance engines



4. Lack of testing on latest serving engines

5. It is not yet fully generalized or optimized for the broader landscape of language model serving engines

Ctrl C + Ctrl V Crew

Credits:

Rishit
Shubham
Trisha
Shiveish
Chaitanya
Ishwin
Hrishita
Arushi
Vanshika